

Relatório 23 - Reconhecimento de Emoções com TensorFlow 2.0 e Python (III)

Felipe Fonseca

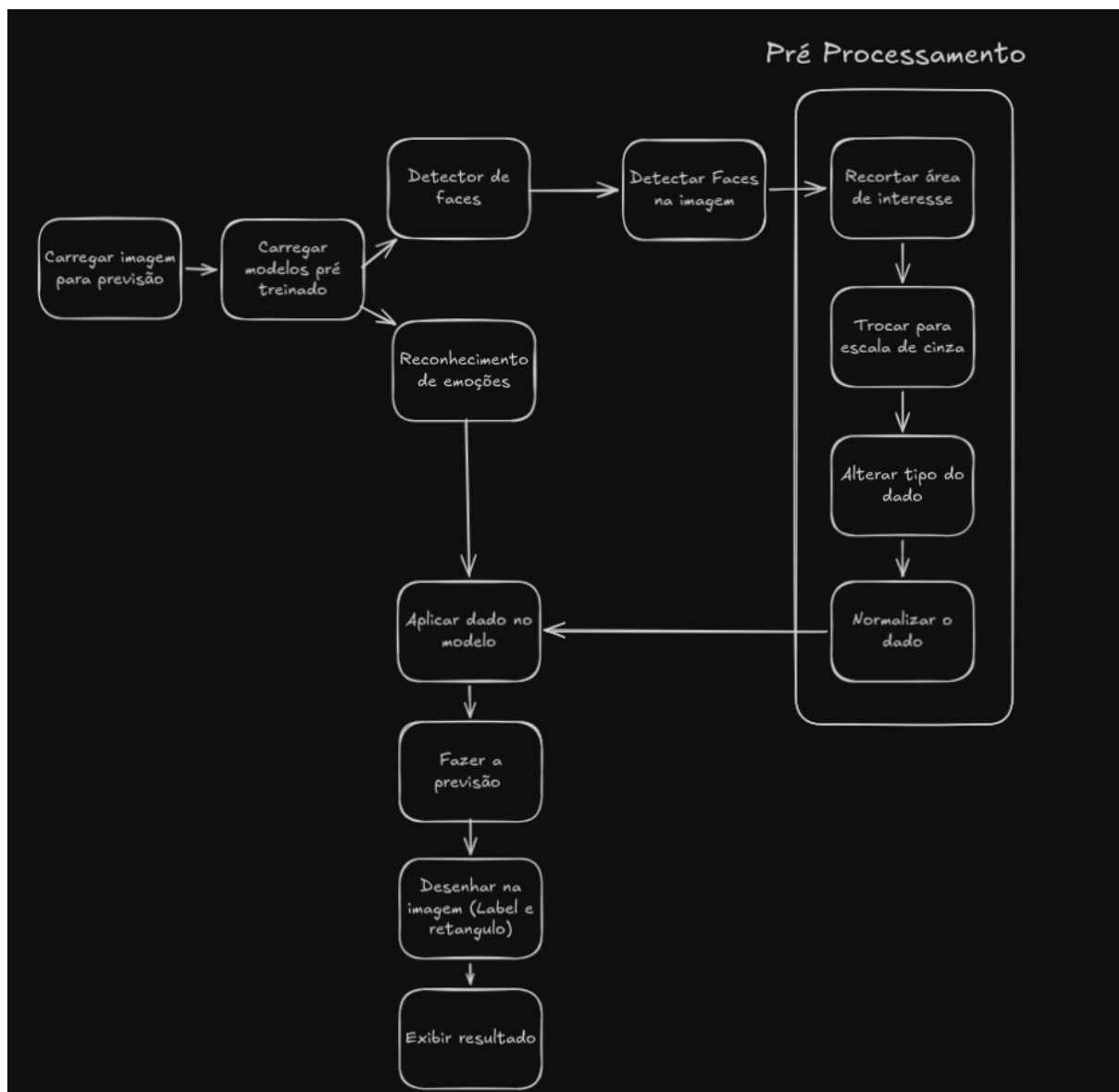
1. Introdução

Nesse card foram desenvolvidas atividades com reconhecimento de emoções utilizando a biblioteca do tensor flow e do opencv. O foco se dividiu em preparar os dados do dataset FER2013, criar o modelo da rede neural para fazer o treinamento, definir novas táticas de treinamento como os callbacks, e então analisar os resultados. Também foram feitos testes com outros modelos e definidos alguns conceitos importantes para o trabalho com redes neurais.

2. Desenvolvimento

2.1 Ideia inicial

Reconhecimento de Emoções: é a tarefa de analisar imagens ou vídeos e prever qual a emoção predominante de uma pessoa. Seu objetivo principal é melhorar a comunicação entre humano e computador, área conhecida como computação afetiva. As aplicações incluem carros inteligentes, jogos, saúde, educação, vigilância, marketing e cinema. A abordagem utilizada para isso é baseada em Redes Neurais Convolucionais (CNNs).

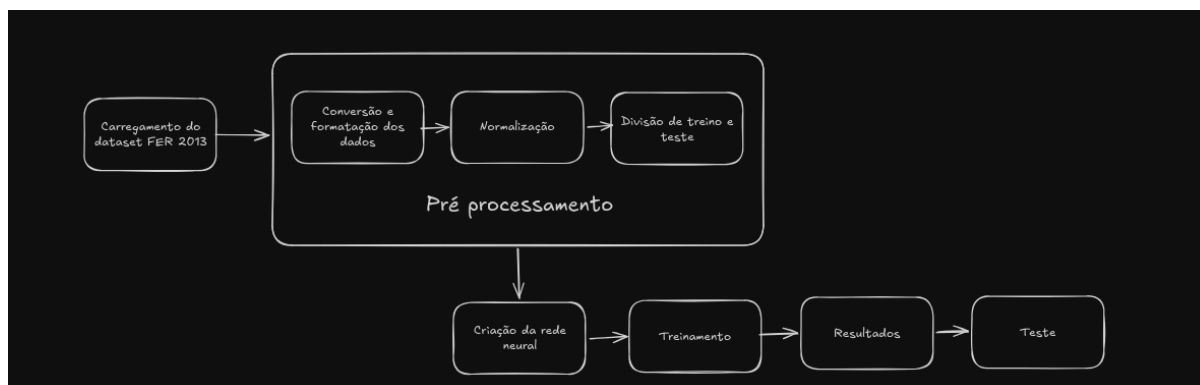


2.2 Modelos Pré-Treinados e Testes

Inicialmente, o foco foi a utilização de modelos já treinados para demonstrar o reconhecimento de emoções na prática, com grande uso da biblioteca cv2 para

manipular e exibir as imagens. O fluxo de uso consistiu em: carregar os modelos pré-treinados, utilizar um modelo de detecção facial (também pré-treinado) para identificar e recortar a área de interesse na imagem, aplicar as funções de tratamento necessárias (conversão de cor, tipo de dado, normalização) e então realizar a previsão da emoção. O resultado foi exibido com o cv2, desenhando um retângulo sobre a face detectada com a label da emoção prevista.

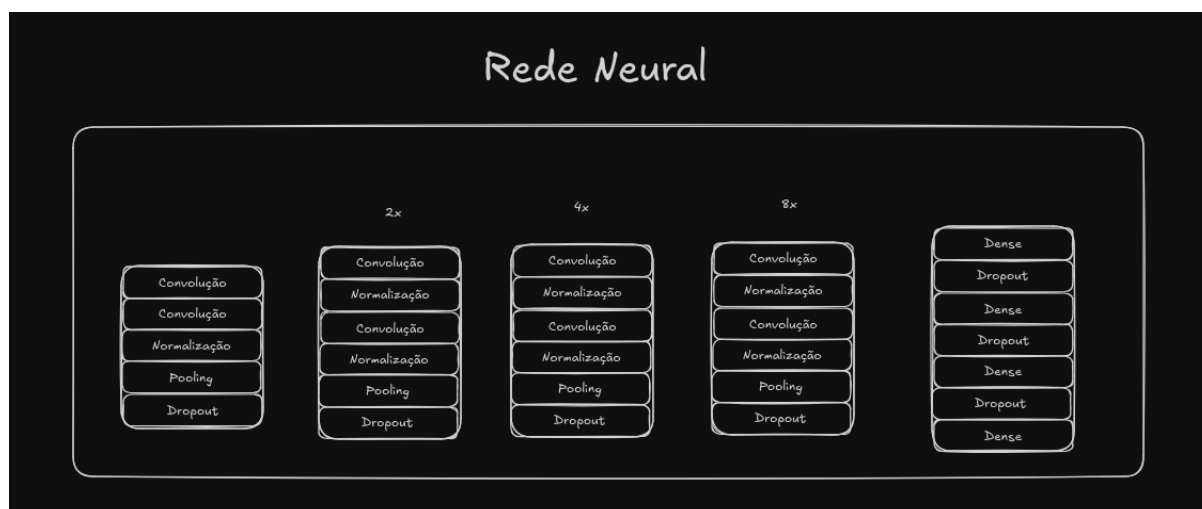
2.3 Criação e treinamento do Modelo



Pré-processamento dos Dados: O dataset utilizado foi o FER2013. Os dados brutos passaram por vários tratamentos: conversão para arrays NumPy, formatação nas dimensões corretas da matriz, normalização dos valores e transformação dos identificadores de emoção em vetores one-hot (arrays com a probabilidade para cada uma das 7 classes de emoção). Por fim, os dados foram separados em conjuntos de treino e teste.

Uma dificuldade encontrada foi o uso do imshow no Jupyter Notebook pelo VS Code, diferente do ambiente Google Colab mostrado pelo instrutor. A solução foi utilizar a função imshow do cv2, junto com cv2.waitKey para garantir que as imagens carregassem corretamente, e cv2.destroyAllWindows ao final para fechar as janelas abertas.

Arquitetura da CNN: A rede foi construída com a biblioteca TensorFlow, com os seguintes hiperparâmetros principais: 64 mapas de características, 7 classes de saída, batch size de 64 e 100 épocas de treinamento. O formato dos dados utiliza channels last, indicando que o canal de cor (escala de cinza, canal único) fica na última posição do tensor.



A rede é organizada em 4 blocos de camadas. O primeiro bloco contém: uma camada de convolução com padding same (para não reduzir as dimensões da imagem), uma camada de normalização em lote, uma camada de pooling e uma camada de dropout com valor 0.5. O dropout elevado foi escolhido deliberadamente para combater o overfitting ainda nas primeiras camadas.

Os blocos 2, 3 e 4 seguem um padrão semelhante — camadas de convolução intercaladas com normalização, seguidas de Max Pooling e Dropout ao final — com uma diferença importante: a cada novo bloco, o número de filtros de convolução é dobrado. Isso permite que a rede extraia características mais complexas de forma progressiva.

Em todas as camadas de convolução foi aplicado o regularizador L2 (kernel regularizer), que penaliza pesos muito altos, forçando-os a permanecer mais uniformes. Essa técnica é útil para evitar que os pesos se ajustem excessivamente aos dados de treino, ou seja, evitar o overfitting.

Para finalizar a arquitetura, foi adicionada uma camada de Flatten, seguida por camadas densas que começam com muitos neurônios e vão reduzindo pela metade progressivamente, até se aproximar do número de classes de saída.

Preparação para o Treinamento: Antes de iniciar o treinamento, três callbacks foram configurados para tornar o processo mais eficiente e controlado:

ReduceLROnPlateau: Reduz a taxa de aprendizado automaticamente caso o modelo pare de evoluir, monitorando a `val_loss` com fator 0.9 e paciência de 3 épocas. Isso evita que uma taxa de aprendizado fixa trave o modelo em um patamar sem conseguir progredir.

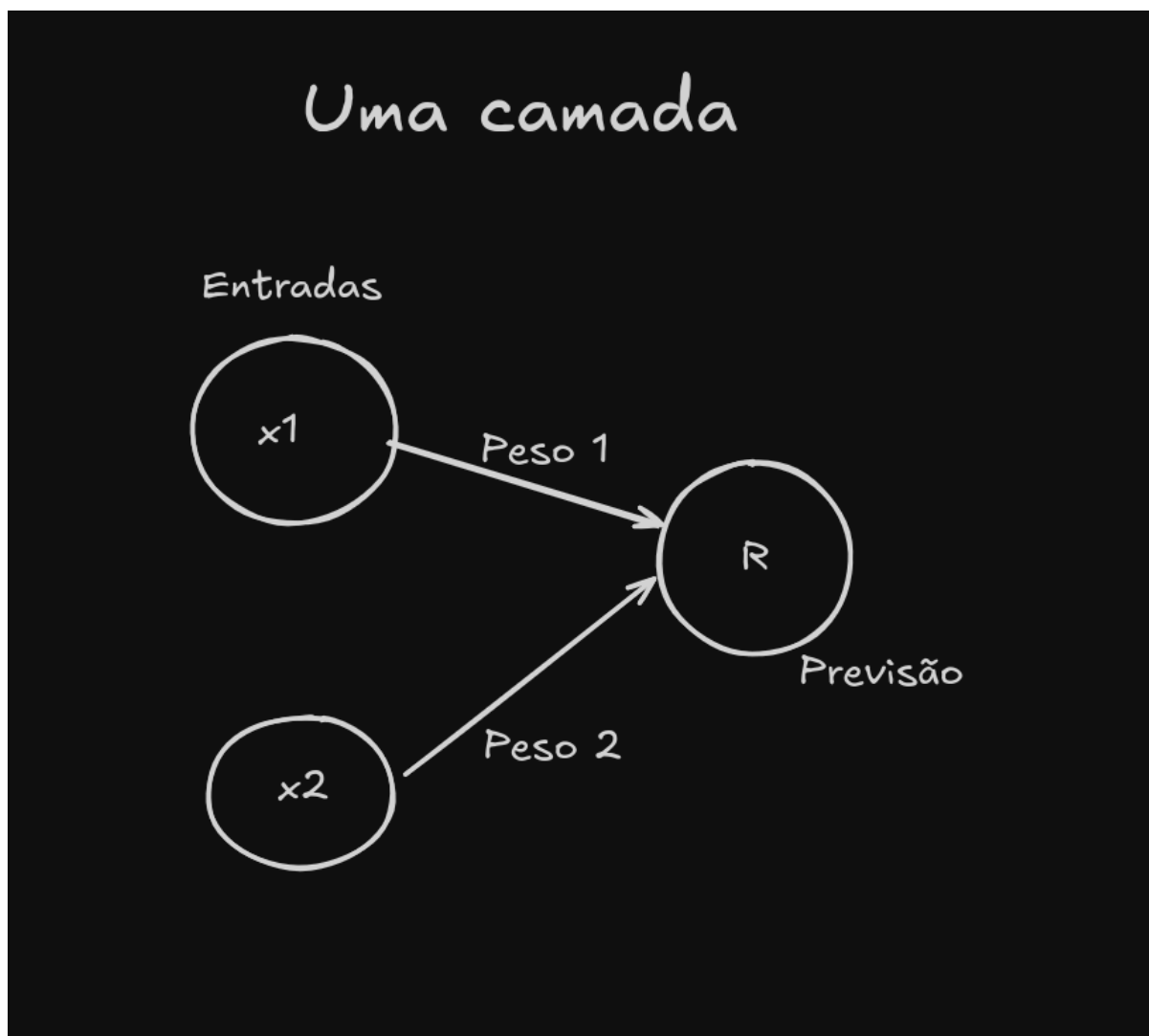
Early Stopping: Interrompe o treinamento caso o modelo pare de aprender, com paciência de 8 épocas monitorando a `val_loss`. Evita desperdício de processamento e previne overfitting por treinamento excessivo.

ModelCheckpoint: Salva automaticamente os pesos do modelo em arquivo sempre que uma melhora é registrada na `val_loss`, garantindo que o melhor estado do modelo seja preservado independentemente de quando o treinamento for interrompido.

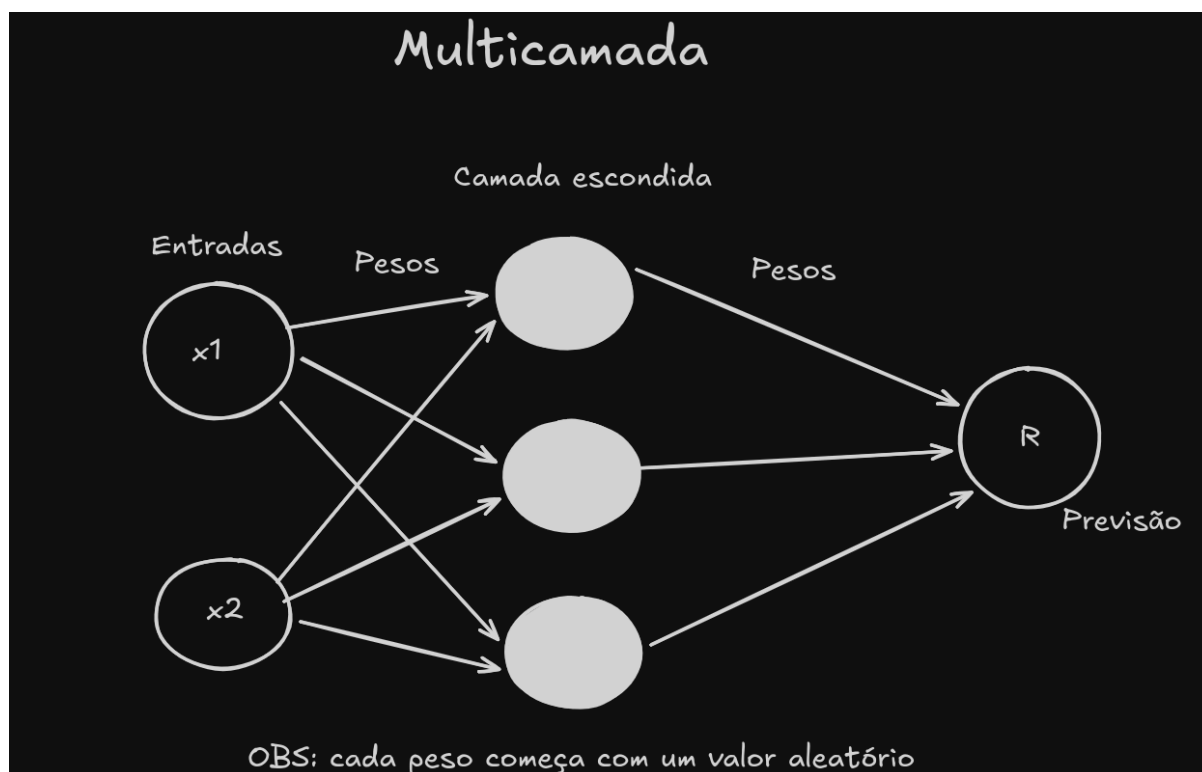
Depois foi realizado um teste comparativo entre modelos com diferentes arquiteturas, onde o fluxo consistia em carregar cada modelo, rodar o evaluate para coletar as métricas e identificar o melhor modelo para um teste final com uma outra imagem. E também foi feito um teste com o modelo, mostrando como seria aplicado o cv2 em um reconhecimento de emoções em vídeo invés de uma imagem.

2.4 Teorias:

Depois dessa parte prática, o card volta a ser mais teórico, mas focando em conteúdos que já foram vistos em cards anteriores. Como o funcionamento do perceptron de uma camada:



Ou então o perceptron multicamada:



Foi explicado novamente o cálculo do erro, sendo o mais simples para utilização a fórmula do erro médio absoluto, na qual é feita a diferença absoluta entre a resposta correta e a resposta prevista, e depois a média dessas diferenças.

Também foi explicado novamente a busca pela descida do gradiente utilizando a derivada da função, o cálculo do parâmetro delta e o ajuste do peso com o algoritmo de backpropagation, tudo coisas que já haviam sido vistas anteriormente.

E por fim, explicou de novo os parâmetros obrigatórios de conhecer como:

- Learning rate: taxa de aprendizado, ou seja, o quão rápido o ajuste dos pesos é feito.
- Batch Size: tamanho do lote, ou seja, a quantidade de registros que será enviado por vez para a atualização dos pesos.
- Epochs: épocas, ou seja, quantas vezes devemos fazer a atualização dos pesos.

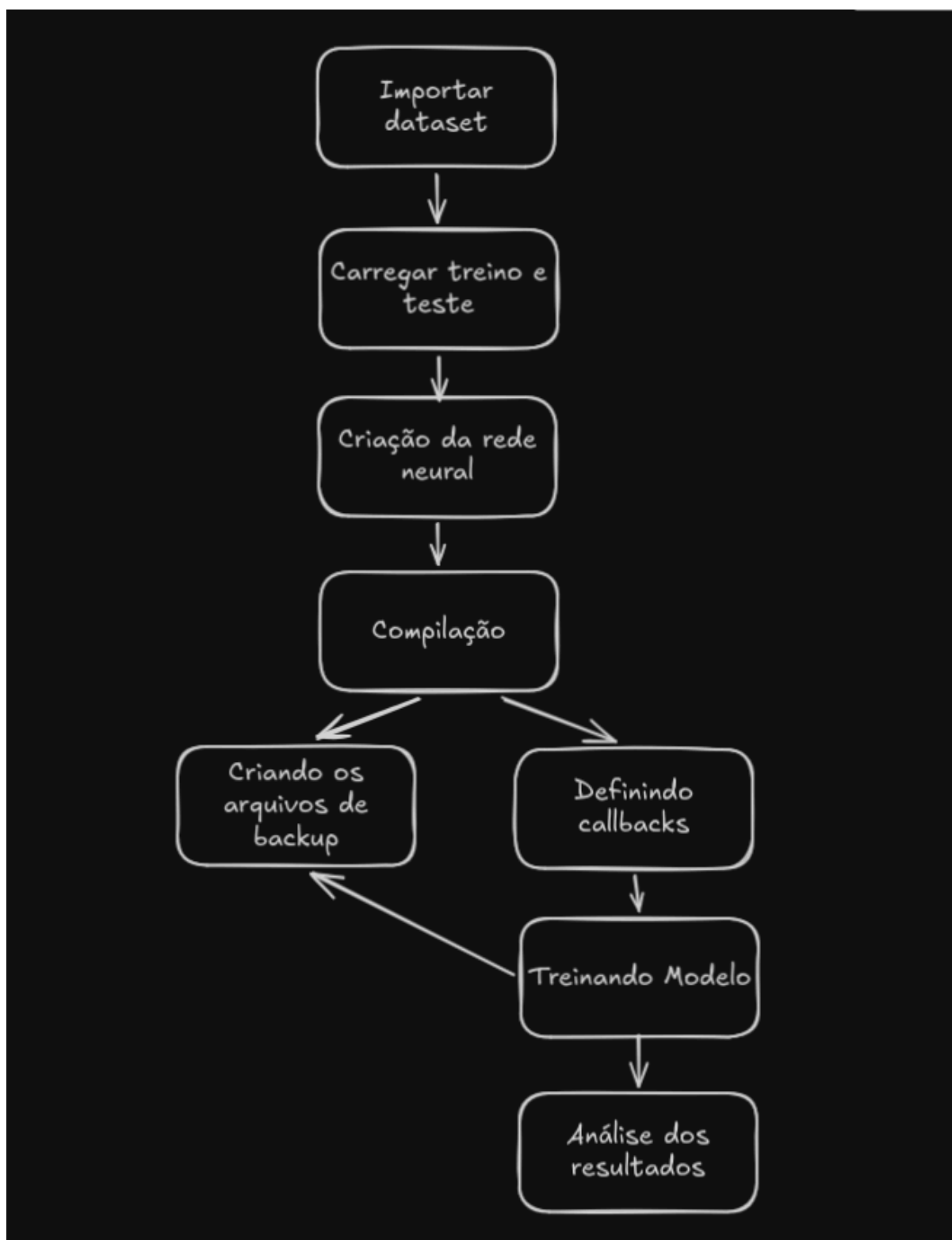
E as principais funções de ativação:

- Step Funcion (função degrau): retorna 0 para valores negativos e 1 para valores positivos.
- Função sigmoide: retorna valores entre 0 e 1.

- Função tangente hiperbólica: retorna valores entre -1 e 1.
- Função relu (a mais utilizada nas camadas ocultas): retorna 0 para valores negativos, e retorna o próprio valor para valores positivos.
- Função softmax (ideal para classificação): retorna uma probabilidade para cada possível resposta.
- Função linear(ideal para regressão): retorna o próprio valor.

2.5 Prática

Na prática eu fiz utilização do google colabs para que o treinamento fosse mais rápido.



Importação do dataset feita pelo kaggle, o dataset utilizado foi o mesmo das aulas, o FER2013;

Carregamento dos dados feitos através do `image_dataset_from_directory`, método do Keras que já carrega todas as imagens para um arquivo, já colocando as pastas como as categorias de respostas, podendo especificar o modo de cor entre outras

variáveis. Foi utilizada pois acelera muito o processo por não ter que fazer a separação e pré processamento dos dados de forma manual.

A criação da rede neural teve apenas algumas diferenças na estrutura em relação a aula, com foco em testar algumas camadas diferentes. Em específico, cito a LeakyRelu, que é uma camada que invés de zerar qualquer valor menor que 0, multiplica esse valor por um valor muito pequeno, evitando que fiquem muitos valores nulos nos pesos. E o outro teste que eu fiz, foi colocar o GlobalAveragePooling2D invés de uma camada de flatten, o que ela faz é, invés de transformar a matriz em um array, ela faz a média dos valores dos filtros de características, deixando o processamento mais leve, mas ao mesmo tempo podendo perder características importantes.

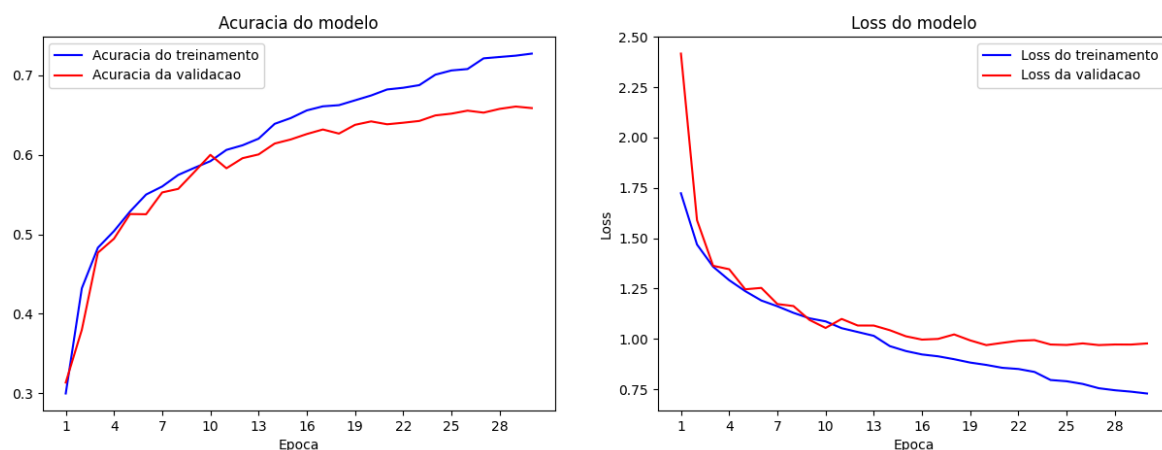
Para compilação foi feito igual a aula, utilizando o adam personalizado com learning rate baixo, e um valor epsilon bem pequeno.

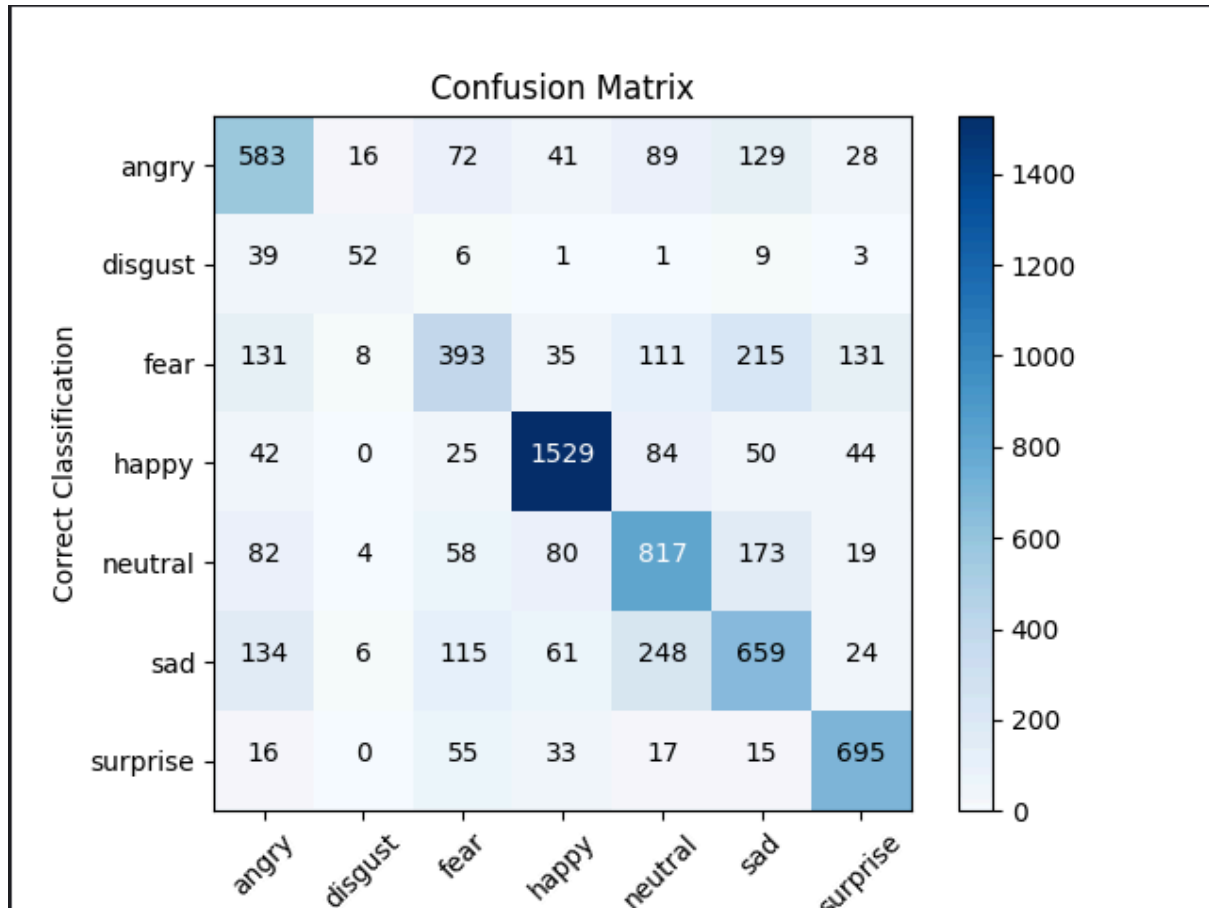
Como a primeira parte foi feita no colabs, a parte de salvar os arquivos foi essencial, pois eu precisaria desses arquivos para rodar a segunda parte da prática.

Os callbacks também foram os mesmos das aulas, destacando principalmente o checkpointer que é o callback mais interessante dos 3 testados.

O treinamento foi definido como 100 épocas, porém já na 30 ele parou devido ao valor não ter melhorado.

Quanto à análise de resultados, eu fiz os 2 gráficos com o histórico ao longo do treinamento, e também com a matriz de confusão.





Para a segunda parte da prática, eu fiz um código em python que tenta prever em tempo real a emoção da pessoa com a webcam.

Ele carrega o cascade que foi utilizado nos outros também, e carrega o modelo que eu acabei de salvar. Depois utilizamos o cv2 para pegar os frames da câmera, fazemos a previsão, e desenhamos isso no frame da câmera.

3. Conclusão

Esse card foi muito bom para o aprofundamento na biblioteca do tensorflow e no opencv, uma combinação muito boa que junta uma lib excelente para a criação e carregamento de redes neurais, e uma lib que é muito útil para o tratamento de imagens. Os tópicos que mais chamaram atenção foram os novos aprendizados do tensor flow, como salvar o modelo e poder carregar a qualquer momento, e as formas de utilizar o cv2 para que os resultados fiquem mais interativos com o usuário através de imagens.

4. Referências

Videoaulas do Card

[Documentação do Keras](#)